Nanocode, explained

A chapter-by-chapter walkthrough of a small coding agent, written for students.

This is the companion book to the code in this repository. It assumes you have the code open beside it. Each chapter explains one new idea, shows the code that implements it, walks through what happens when the program runs, and ends with things to try and questions to check yourself against.

The repository is an educational refactor of *Build Your Own Coding Agent*: the chapters and their ideas are the book's, the structure is not. Every chapter's original single file is kept as `nanocode_copy.py`, so you can diff the refactor against the source at any point.

How to use this book

Three ways, all valid:

1. **Cover to cover.** Read chapters 1-14 in order. Each builds on the last, and nothing is assumed except basic Python.
2. **Alongside the code.** Open `chNN/nanocode.py` and the chapter here at the same time. Every section names the file and function it is talking about.
3. **As a reference.** Appendix A is the whole architecture on one page; Appendix B is a glossary; the checkpoint boxes answer the questions students actually ask.

Each chapter has the same shape:

Section	What it is for
You are here	one sentence: what this chapter adds
The idea in one picture	an ASCII diagram you can redraw from memory
What's new on disk	which files changed, and why those files
The code	the real snippets, annotated
Watch it run	a frame-by-frame trace of the program running
Why it is built this way	the trade-offs, stated as trade-offs
Try it	exercises that fail in instructive ways
Checkpoint	collapsible questions with answers

What you need

```
# 1. install uv (https://docs.astral.sh/uv/getting-started/installation/)
curl -LsSf https://astral.sh/uv/install.sh | sh        # macOS / Linux
# powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"   # Windows

# 2. get the code and the dependencies
git clone <this-repo> && cd build-your-own-coding-agent-main
uv sync

# 3. a key for the default brain (OpenRouter), in .env at the repo root
#    OPENROUTER_API_KEY=sk-or-...
#    OPENROUTER_MODEL="deepseek-v4.1-flash-1017"
#    NANOCODE_BRAIN="openrouter"
```

Chapters 1 and 2 need no key. From chapter 3 on, every chapter needs one model provider — OpenRouter by default, or `NANOCODE_BRAIN=claude|deepseek|ollama`.

Conventions used here

- **Diagrams are plain ASCII.** They render identically in Markdown, in `book.html`, in a terminal and on paper — no diagram plugin required.
- `──►` **is data flow,** `▼` **is time moving down.** Boxes are components; the text inside is what is passed between them.
- **Frame-by-frame traces** are real program output, split into "frames". Read them top to bottom; they are the animation.
- **Hook lines** (`✧ hook:tool_call ...`) are printed by this fork, not by the original book. They are on by default; `NANOCODE_HOOKS=0` hides them.
- **Collapsible answers** use HTML `<details>`. In a plain terminal view you will see the tags — read around them.
- **Snippets are excerpts, not photocopies.** Long expressions are sometimes joined onto one line, `...` marks omitted code, and comments are occasionally added for the point being made. The files themselves are the source of truth.

Contents

#	Chapter	The one new idea	Page
1	The event loop	a prompt, an agent, a reply — nothing else	4
2	First contact	one HTTP request, seen in full	7
3	The first brain	`Thought`, and a conversation that remembers	10
4	More than one brain	an interface, a registry, retries	14
5	Hands	tools, and the loop becomes a cycle	18
6	Memory that survives	a file as the system prompt	22
7	Plan mode	safety by construction, not instruction	26
8	Eyes on the project	list and search before acting	30
9	Closing the loop	run it, read the error, fix it; compaction	33
10	Local models	a fourth brain, no network	37
11	A window outside	web search as a tool	40
12	Capstone	the finished agent, and how to read it	44
13	Sound	the same hooks, heard: a Windows sound per event	48
14	Sandbox	ACT mode writes only where the policy allows	53

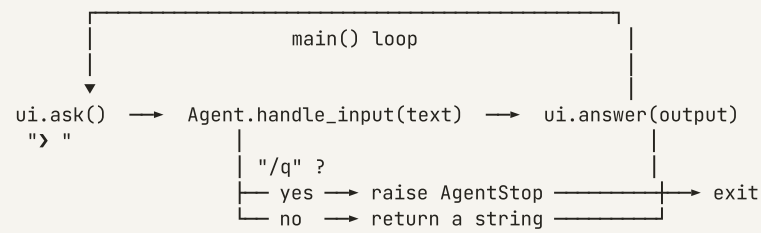
Each chapter also exists as its own file (`chNN/EXPLAINER_chNN.md`); this document is the combined edition.

Chapter 1 — The event loop (there is no AI yet)

> **You are here:** nothing talks to a model. This chapter builds the *shape* every later chapter plugs into: read a line, hand it to an Agent, print what comes back, repeat until the user leaves.

That sounds trivial, and it is — which is exactly why it is chapter one. Every agent you have ever used is this loop with more interesting things happening inside step 2.

The idea in one picture



Four moving parts, and only one of them will grow:

Part	What it does	Does it change later?
`main()`	reads input, prints output	barely
`Agent`	turns input into output	**everything happens here**
`ui.py`	draws the prompt and the answer	grows a lot (colour, Markdown)
`ui.json`	the colours	grows a lot

What's new on disk

```
ch01/
  nanocode.py   AgentStop, Agent, main()      ← the whole chapter
  ui.py         Rich wrappers: banner, ask, answer
  ui.json       every colour, as data
  test_nanocode.py  the book's tests (4 of them)
```

`Agent` is deliberately empty of intelligence. It records nothing, remembers nothing, and calls nothing:

```
class Agent:
    """A coding agent that processes user input."""

    def __init__(self):
        pass

    def handle_input(self, user_input):
        if user_input.strip() == "/q":
            raise AgentStop()

        if not user_input.strip():
            return ""

        # Chapter 1 has no model behind the curtain. Returning the input proves
        # the plumbing works: something reads, something answers, something draws.
        return f"You said: {user_input}\n(Agent not yet connected)"
```

Watch it run (frame by frame)

Each frame is one pass around the loop:

```
frame 1                    frame 2                    frame 3
┌──────────────────┐       ┌──────────────────┐       ┌──────────────┐
│ › hi             │       │ › /q             │       │ Exiting...   │
│ You said: hi     │       │ (AgentStop raised)│      │              │
│ (Agent not ...)  │       │                  │       │              │
└──────────────────┘       └──────────────────┘       └──────────────┘
```

The real run looks like this — note that the prompt shows no brain and no mode, because this chapter has neither:

```
 Nanocode v0.1
  type /q to quit
──────────────────────────────────────────────────────────────

› hi
You said: hi (Agent not yet connected)

› /q
Exiting...
```

Why it is built this way

Quitting is an exception, not a flag. `/q` can be typed at a prompt deep inside a future chapter, and the only code that cares is `main()`, which catches `AgentStop` once. A boolean `running = False` would have to be threaded through every layer.

Empty input returns `""`, not a message. `main()` only prints when there is something to print, so pressing Enter does not produce a stray blank line.

All printing goes through `ui.py`. The book uses `print()` here. This version calls `ui.answer(...)` so that later chapters can restyle output without touching the loop — the loop should not know what a colour is.

Try it

```
uv run ch01/nanocode.py
```

Then, in order:

1. Type `hello` and press Enter. Predict the exact output before you do.
2. Press Enter on an empty line. How many lines get printed? Why?
3. Type `/q hej` (with trailing text). Does it quit? Look at the `==` comparison in `handle_input` and explain what you see.
4. Open `ui.json`, change `banner.title`'s `style` to `"bold magenta"`, restart. You have just re-themed the app without editing Python.

Checkpoint

<details>
<summary>Why does `handle_input` return a string instead of printing?</summary>
Because "produce the answer" and "draw the answer" are different jobs. Returning a string means the agent can be tested without a terminal, and `main()` decides how the answer appears. Every later chapter keeps this split: `Agent` returns, `ui` renders.
</details>

<details>
<summary>What would break if `Agent` printed directly?</summary>
The tests. `test_nanocode.py` calls `handle_input` and asserts on the *return value*; printed text would be invisible to them. It would also duplicate output the moment a chapter wants to render Markdown instead of plain text.
</details>

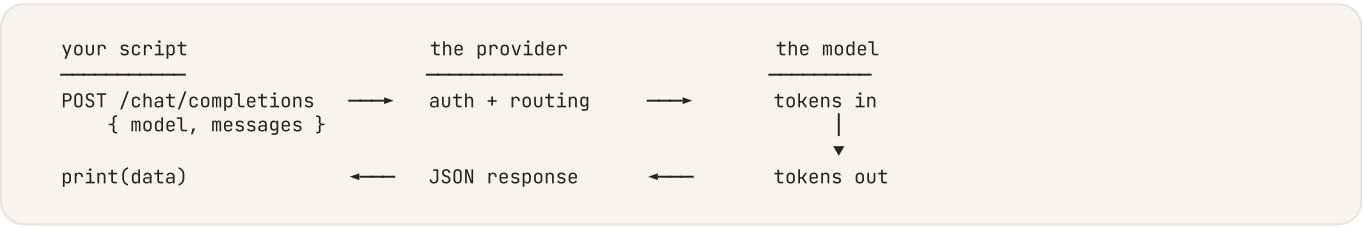
Next: chapter 2 makes one real HTTP request to a model — still with no agent.

Chapter 2 — First contact (one raw request)

> **You are here:** still no agent. This chapter sends a single HTTP request to a model and prints the JSON that comes back, so you can see exactly what a "brain" looks like before any abstraction hides it.

The book does this with Claude. This fork does it with OpenRouter, because the same key then drives every later chapter. The mechanics are identical — a URL, a key, a JSON body, a JSON reply.

The idea in one picture



No loop, no tools, no memory. If you understand this arrow diagram, every chapter after this one is bookkeeping.

What's new on disk

```
ch02/
  api_check.py    the ported script (this is what you read)
  test_api.py     the book's original, untouched, for comparison
  ui.py, ui.json  same console layer as chapter 1
```

`test_api.py` is the book's version and it calls Anthropic directly. It is kept so you can diff the two dialects side by side:

```
# book (test_api.py)                    # this fork (api_check.py)
"x-api-key": api_key,                   "Authorization": f"Bearer {api_key}",
url = ".../v1/messages"                 url = ".../v1/chat/completions",
payload["messages"] = [...]             payload["messages"] = [...]      # same
```

The request, field by field

```
headers = {
    "Authorization": f"Bearer {api_key}",   # who you are
    "Content-Type": "application/json",     # what you are sending
    "X-Title": "Nanocode",                  # shows up in OpenRouter's dashboard
}

payload = {
    "model": model,        # which model; overridable with OPENROUTER_MODEL
    "max_tokens": 256,     # budget for the REPLY, not the request
    "messages": [          # the conversation so far (here: one message)
        {"role": "user", "content": "Hello, are you ready to code?"}
    ],
}
```

The response is bigger than you expect. Two fields matter now:

```
choices[0].message.content    ← what the model said
usage.prompt_tokens           ← what the request cost (used in chapter 9)
```

Everything else — `id`, `provider`, `service_tier`, `system_fingerprint` — is provider bookkeeping. It is printed, syntax-highlighted, because seeing it once makes the later abstractions feel less magical.

Watch it run (frame by frame)

```
  · Sending request...            ← before the network call
  · Status: 200                   ← the HTTP status nobody checks until it is 500
  ✓ Ready when you are! ...       ← the one field a user cares about

  {                               ← everything else, for the curious
    "id": "gen-...",
    "model": "deepseek/deepseek-v4.1-flash-1017",
    "provider": "Wafer",
    "choices": [ { "message": { "content": "Ready when you are! ..." } } ],
    "usage": { "prompt_tokens": 36, "completion_tokens": 12, "total_tokens": 48 }
  }
```

Why it is built this way

No error handling on purpose (mostly). The book fires the request "with no safety net". Chapter 4 adds retries. Here, a 401 should be loud — you are meant to see what a failure looks like before you start hiding failures.

The JSON is printed, not summarised. An abstraction you have not seen the insides of is a superstition. Once you have read `choices[0].message`, chapter 3's `Thought` object reads like a translation, not a trick.

The key never appears in output. `ui.error` prints the provider's message on failure, not the request — a small habit worth copying.

Try it

```
uv run ch02/api_check.py
```

1. Run it twice. Which fields change between runs? Which stay the same?
2. Break the key on purpose (add a letter in `.env`), run again, and read the error. What status code comes back, and where does the message live in the JSON?
3. Change `max_tokens` to `8`. What happens to the reply, and what is `finish_reason` now? (Add a print if it is not shown.)
4. Change the prompt to ask for a 5-item Markdown list. Notice the JSON string contains `\n` escapes — this is why chapter 3 stores text, not formatted output.

Checkpoint

<details>
<summary>Why is <code>max_tokens</code> in the request if the model already knows when to stop?</summary>
Because it caps cost and latency. A model will happily write 2000 tokens when 20 would do; `max_tokens` is your budget, and hitting it shows up as `finish_reason: "length"` — a truncated reply, not an error.
</details>

<details>
<summary>What does the model actually receive from you?</summary>
The messages array, serialised as JSON, plus the model name. No files, no memory, no state: everything the model knows about your project is whatever you literally put in those messages. That fact is the whole reason tools exist.
</details>

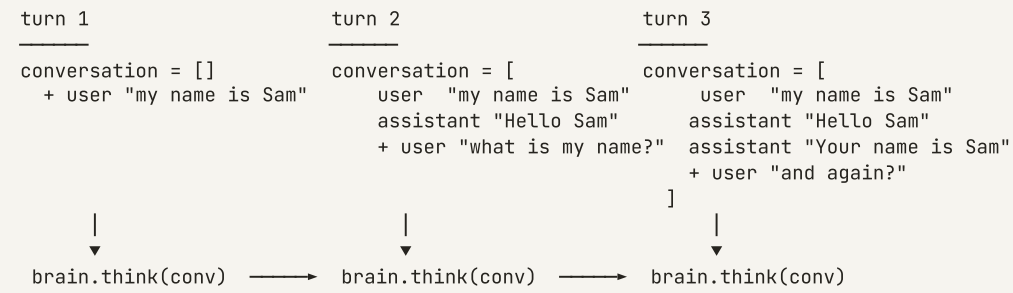
Next: chapter 3 wraps this request in a class, keeps the messages around, and gives the agent its first memory.

Chapter 3 — The first brain (and the conversation becomes memory)

> **You are here:** the request from chapter 2 becomes a class, and the class's output becomes a `Thought`. The agent now remembers what you said — because it resends the whole conversation every turn.

This is the first chapter where behaviour feels like magic. It is not. The model has no memory; the *list* has the memory, and you send it again and again.

The idea in one picture



The arrow into `brain.think` always carries *everything*. That is the memory.

What's new on disk

```
ch03/
  llm.py       Thought, ToolCall, OpenRouter      ← the brain, in its own file
  nanocode.py  Agent gains a conversation list
  hooks.py     lifecycle events (this fork's addition)
  ui.py        adds thinking lines
```

Three new ideas, in order of importance:

Idea	Where	Why it matters
`Thought`	`llm.py`	one shape for "what the model said", whatever the provider
`conversation`	`nanocode.py`	the entire state of the agent
hooks	`hooks.py`	the loop can be observed without being edited

The brain, and the object it returns

```
class OpenRouter:
    def think(self, conversation):
        headers = {...}
        payload = {"model": self.model, "max_tokens": 4096, "messages": conversation}
        response = requests.post(self.url, headers=headers, json=payload, timeout=120)
        response.raise_for_status()
        return self._parse_response(response.json()["choices"][0]["message"])
```

The important line is the last one: the raw provider JSON never leaves this class. The agent receives a `Thought`:

```
class Thought:
    def __init__(self, text=None, tool_calls=None, thinking=None):
        self.text = text              # what to show the user
        self.tool_calls = tool_calls or []  # empty here; tools arrive in ch05
        self.thinking = thinking      # reasoning, for the dim 💭 lines only
```

That empty `tool_calls` list is a promise being made early: the loop already knows it might be asked to run something.

The agent, and the list that remembers

```
self.conversation.append({"role": "user", "content": user_input})

try:
    thought = self.brain.think(self.conversation)
    ui.thinking(thought.thinking)

    text = thought.text or ""
    self.conversation.append({"role": "assistant", "content": text})
    return text
except Exception as e:
    self.conversation.pop()   # the question never got an answer: drop it
```

Two details students usually miss:

1. **The reply is appended too.** If you only append user messages, the next request contains a conversation where the model talks to itself.
2. **A failed call pops the user message.** Otherwise the history has two user turns in a row, and some providers reject that outright.

Watch it run (frame by frame)

Real output, with the reasoning lines this fork adds:

```
[openrouter] › my name is Sam
  💭 The user gave their name. Acknowledge it briefly.
Hello, Sam! Nice to meet you.

[openrouter] › what is my name?
  💭 We know from the prior turn: Sam. Answer concisely.
Your name is Sam.
```

Now look at the hook lines that this fork prints (they are on by default; set `NANOCODE_HOOKS=0` to hide them):

```
[openrouter] ›   ◇ hook:turn_start  "what is my name?"
  ◇ hook:before_think  3 message(s) queued        ← 3, because history is resent
  💭 We know from the prior turn: Sam.
  ◇ hook:after_think  text 18 chars
  ◇ hook:turn_end  answer 18 chars
```

`3 message(s)` is the whole lesson of this chapter: user, assistant, user.

Why it is built this way

The brain moved to `llm.py` even though there is only one provider. Chapter 4 adds two more; moving a class while it is still small is cheap, moving it later is a refactor.

`Thought` **has exactly three fields.** Every one of them is used by the agent or the UI this chapter. Fields are added when a chapter needs them (`raw_content` in ch05, token counts in ch09) — not in advance.

Hooks exist from the first brain onwards. They are the cheapest way to make the loop legible: five `emit` calls in the agent, and any listener can watch.

Try it

```
uv run ch03/nanocode.py
```

1. Tell it your name, then ask for it. Then ask again after three unrelated questions. Does it still know? Look at how many messages were sent (`hook:before_think`).
2. Ask it something that needs your project: "what is in this repo?" It cannot know — it has never seen a file. Remember that feeling; chapter 5 fixes it.
3. Register a hook without editing the agent:

```
import nanocode
from hooks import Hooks

agent = nanocode.Agent(brain=nanocode.OpenRouter())
agent.hooks.on("turn_end", lambda output, **_: print("ANSWER LEN:", len(output)))
```

Four lines, no changes to `nanocode.py` . That is what "observable" buys you.

Checkpoint

<details>
<summary>Where does the model's "memory" actually live?</summary>
In `Agent.conversation` — a plain Python list on your machine. The model is stateless between requests; resending the list is what feels like memory. This is also why long conversations get expensive: every turn resends everything.
</details>

<details>
<summary>Why does a failed call remove the user message?</summary>
A conversation with a dangling user question has the wrong shape: two user messages in a row on the next attempt. Removing it keeps the alternation user/assistant/user that providers expect.
</details>

<details>
<summary>What is <code>thinking</code> for, if it is not sent back?</summary>
It is shown once, dimmed, then dropped. Reasoning is produced by the model, but you do not need to replay it — the answer (and later the tool calls) carry the information forward. Keeping it would inflate every subsequent request.
</details>

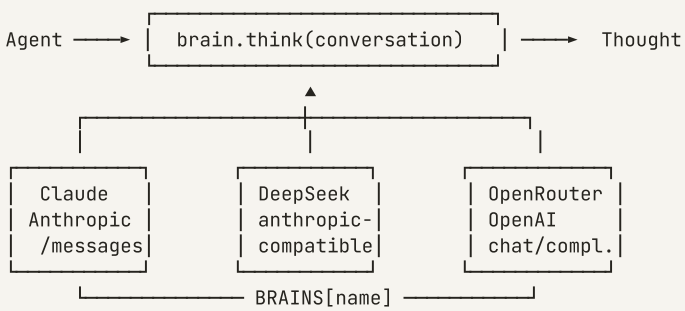
Next: chapter 4 adds a second and third provider, a retry policy, and `/switch` — without touching the agent loop.

Chapter 4 — More than one brain (an interface, a registry, retries)

> **You are here:** the agent stops knowing which model answers it. A `Brain` is now a contract, providers are implementations, and `/switch` swaps them mid-conversation.

The interesting thing about this chapter is how little the agent changed. Adding two providers added zero lines of loop code. That is what an abstraction is for — not tidiness, but *leverage*.

The idea in one picture



The agent only ever sees the arrow on the left. Everything on the right is replaceable.

What's new on disk

```
ch04/
  llm.py       Brain (base class), Claude, DeepSeek, request_with_retry, BRAINS
  nanocode.py  Agent gains _switch_brain() and a brain_name
```

New thing	Purpose	Where
`Brain`	the contract: implement `think()`, share `_parse_response()`	`llm.py`
`request_with_retry`	survive 429s and 5xx without losing the turn	`llm.py`
`BRAINS`	name → class, the list `/switch` walks	`llm.py`
`_switch_brain`	swap the object, keep the history	`nanocode.py`

The contract, and why a base class earns its keep

```
class Brain:
    """Base class for LLM providers.

    Two methods matter: think() (which every provider must implement) and
    _parse_response() (shared by the Anthropic-shaped providers).
    """

    def think(self, conversation):
        raise NotImplementedError

    def _parse_response(self, content):
        # Anthropic-shaped providers all return a list of blocks: thinking,
        # text, tool_use. This conversion is written once and shared.
        ...
```

`Claude` and `DeepSeek` differ only in URL, headers and payload — the parsing is identical, so it lives in the base class. `OpenRouter` speaks a different dialect and overrides `_parse_response`. The base class is doing exactly one job: naming the shared parts.

Retries: three cases, three behaviours

```
requests raises RequestException        → network died: wait 2**attempt, retry
status 429 or ⩾ 500                     → server busy: wait Retry-After (or 2**attempt), retry
status ⩾ 400                            → our request is wrong: raise, do not retry
```

The distinction matters: a 429 is "later", a 401 is "never". Retrying the second one just wastes ten seconds before failing anyway.

`/switch` — and why the history survives

```
names = list(BRAINS.keys())             # cycle order = dict order
idx = names.index(self.brain_name)
new_name = names[(idx + 1) % len(names)]

self.brain = BRAINS[new_name]()         # construct inside try: no key → ValueError
self.brain_name = new_name
```

At this chapter the conversation is plain `{"role", "content"}` text, so every provider can read it and nothing has to be translated. Chapter 5 changes that: tool calls force the two dialects apart, and `llm.py` grows translation functions.

Watch it run (frame by frame)

```
[openrouter] ❯ explain a monad in one line
  🧠 Keep it short...
A monad is a way to chain operations while carrying context.

[openrouter] ❯ /switch
  ⇄ Switched to: claude (claude-sonnet-4-6)        ← or an error if no key
[claude]     ❯ and again, shorter
  ...                                               ← same history, new provider
```

If the next provider has no API key, the switch fails *politely*:

```
  ✗ Cannot switch to claude: ANTHROPIC_API_KEY not found in .env
```

The agent keeps running. A provider you cannot afford should not end your session.

Why it is built this way

The registry is an ordered dict, not a list of tuples. `/switch` needs an order and `main()` needs a lookup; one structure serves both.

Construction happens inside the `try`. A missing key raises `ValueError` at `__init__` time, so the switch must be able to fail without corrupting the agent.

The base class holds no state. Its `context_limit` / `last_input_tokens` are placeholders for chapter 9; a provider is otherwise a pure request builder.

Try it

```
uv run ch04/nanocode.py
```

1. Ask a question, `/switch`, ask "what did I just ask?". Does the new provider know? Why?
2. `/switch` twice. Where do you end up? (Count the entries in `BRAINS`.)
3. Set `NANOCODE_BRAIN=claude` with no Anthropic key. Read the error and note *where* it is raised (`main()` catches it, not the agent).
4. Add a fake provider in the REPL:

```
import nanocode
class Echo(nanocode.Brain):
    def think(self, conversation):
        return nanocode.Thought(text=f"echo: {conversation[-1]['content']}")
nanocode.BRAINS["echo"] = Echo
# then /switch until you reach "echo"
```

Checkpoint

▾ Why not put the retry loop inside each provider?

Because it is the same retry loop. Duplicating it three times means three places to fix when a provider adds a new status code. Shared code belongs where the providers can share it — a module-level function, not a base method, because it does not need `self`.

▾ What breaks if you switch brains mid-tool-call (after chapter 5)?

The message history holds provider-shaped content blocks. This fork solves it by storing history in *one* canonical shape (Anthropic blocks) and translating per provider at send time — see chapter 5's `to_openai_messages`.

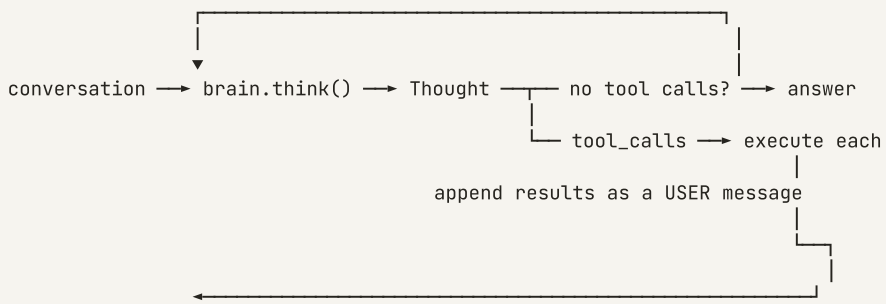
Next: chapter 5 gives the agent hands. The loop stops being a single request and becomes a cycle.

Chapter 5 — Hands (tools, and the loop becomes a cycle)

> **You are here:** the agent can read and write files. More importantly, the single request from chapter 3 becomes a *cycle*: ask → act → feed the result back → ask again.

This is the chapter where "chatbot" becomes "agent". Everything after this is more tools, more safety, more endurance — not a new idea.

The idea in one picture



One trip around that loop is one "iteration". `MAX_ITERATIONS = 50` in the capstone exists because a confused model can keep asking for tools forever.

What's new on disk

```
ch05/
  tools.py     ReadFile, WriteFile, get_tool, tool_definitions
  llm.py       Thought.raw_content, tool_calls, to_openai_messages(), to_openai_tools()
  nanocode.py  _agentic_loop(), _execute_tool()
  ui.py        ui.tool_action() — one coloured line per tool call
```

New thing	Why it exists
tool `execute(**args) -> str`	a tool is a function with metadata the model can read
`raw_content`	the next request must echo exactly what the model said
`to_openai_*`	OpenAI needs a different message shape than Anthropic
`_agentic_loop`	the cycle above, in twenty lines

A tool is metadata plus a function

```
class ReadFile:
    name = "read_file"
    description = "Reads a file from the filesystem. Use this to examine code."
    input_schema = {
        "type": "object",
        "properties": {"path": {"type": "string", "description": "The path to the file"}},
        "required": ["path"],
    }

    def execute(self, path):
        ui.tool_action("read_file", path)
        try:
            with open(path, "r", encoding="utf-8") as f:
                lines = f.readlines()
            numbered_lines = [f"{i+1} | {line}" for i, line in enumerate(lines)]
            return "".join(numbered_lines)
        except Exception as e:
            # A tool must always return a string. The model reads the error
            # and decides what to do next.
            return f"Error reading file: {e}"
```

Four things to notice:

- `description` and `input_schema` are **prompt engineering**, not documentation. They are pasted into the request; the model chooses tools by reading them.
- `execute` never raises. A traceback ends the turn; `"Error: File not found"` continues it.
- The line numbers (`1 | import os`) are for the model, so it can refer to "line 42" later.
- `ui.tool_action(...)` is the only console call in `tools.py`. Colour lives in `ui.json`, so a tool never mentions a colour.

The loop, annotated

```
while True:
    thought = self.brain.think(self.conversation)          # 1. ask
    ui.thinking(thought.thinking)

    # Append the provider's own blocks, not thought.text: the next request must
    # see the tool_use ids that the results will refer to.
    self.conversation.append({"role": "assistant", "content": thought.raw_content})

    if thought.text:
        output_parts.append(thought.text)

    if not thought.tool_calls:
        break                                              # 2. done

    tool_results = []
    for tool_call in thought.tool_calls:                   # 3. act
        result = self._execute_tool(tool_call.name, tool_call.args)
        tool_results.append({
            "type": "tool_result",
            "tool_use_id": tool_call.id,                   # ties result → call
            "content": result,
        })

    self.conversation.append({"role": "user", "content": tool_results})  # 4. feed back
```

Step 4 surprises people: the tool output is appended as a **user** message. In this dialect, "user turn" means "whatever the model needs to hear next", and tool output is exactly that.

One conversation, two dialects

The agent stores history in one shape; providers disagree about the wire format:

```
Anthropic wire format                    OpenAI wire format
─────────────────────                    ──────────────────
{"role":"user",                          {"role":"user",
 "content":[{"type":"tool_result",        "content":"..."}         ← plain text
             "tool_use_id":"c1",         {"role":"assistant",
             "content":"1 | import os"}] "content":null,
                                         "tool_calls":[{"id":"c1",
                                           "function":{"name":"read_file",
                                                       "arguments":"{\"path\":\"app.py\"}"}}]}
                                         {"role":"tool",
                                          "tool_call_id":"c1",
                                          "content":"1 | import os"}
```

`to_openai_messages()` performs that translation on every request. It is the reason `/switch` still works once tools are involved.

Watch it run (frame by frame)

A real two-tool turn, with this fork's hook lines interleaved:

```
[openrouter:act] ❯   ◇ hook:turn_start  "Create note.txt containing hello, then read it back."
  ◇ hook:before_think  1 message(s) queued
  ↑ Thinking...
  ◇ hook:after_think  text 43 chars · wants write_file
  → Writing note.txt
  ◇ hook:tool_call  write_file(path=note.txt, content=hello) → 43 chars back
  ◇ hook:before_think  3 message(s) queued          ← now 3: user, assistant, tool_result
  ↑ Thinking...
  ◇ hook:after_think  wants read_file · reasoning
  💭 Now read it back.
  → Reading note.txt
  ◇ hook:tool_call  read_file(path=note.txt) → 10 chars back
  ◇ hook:before_think  5 message(s) queued          ← 5, and still climbing
  ◇ hook:after_think  text 110 chars
  ◇ hook:turn_end  answer 154 chars
```

The message count is the animation: 1 → 3 → 5. Two tools, two round trips, one answer.

Why it is built this way

Tools return strings, not exceptions. The model cannot catch a traceback; it *can* read "Error: File not found: app.py" and try `app.py` instead. This single decision is what makes the feedback loop work.

`raw_content` **instead of** `text`**.** If you rebuild the assistant message from the text, you lose the tool-call ids, and the tool results refer to nothing.

`MAX_ITERATIONS`**.** An unbounded loop with a paid API is a way to lose money. Fifty is generous for real work and short enough to stop a spiral.

Try it

```
uv run ch05/nanocode.py --act
```

1. "Read `ch05/ui.json` and tell me the colour of `read_file`." Watch which tools it chooses and how many round trips it takes (`hook:before_think`).
2. "Show me the contents of a file that does not exist." Read the error the model gets back, then watch it recover.
3. Ask for something impossible, e.g. "run the tests". It has no such tool. Does it explain, or hallucinate? This is what chapter 9 fixes.
4. In plan mode (the default: no `--act`), `write_file` is still offered in this chapter — plan mode arrives in chapter 7. Try to write a file anyway, then compare with chapter 7.

Checkpoint

▾ **Why does the loop append tool results as a "user" message?**
Because the conversation must alternate. The model produced an assistant message (possibly with tool calls); the results are the next thing it needs to "hear", so they occupy the user slot. Anthropic models know this shape as `tool_result` blocks; OpenAI models use a dedicated `role: "tool"` message instead.

▾ **What happens if you forget `tool_use_id` in a result?**
Providers reject the request or (worse) the model cannot tell which call the result belongs to and re-issues the call. With two tool calls in one reply, the ids are what keep the answers matched to the questions.

▾ **Why are tool descriptions phrased as instructions ("Use this to examine code")?**
They are read by the model when it decides what to call, so they answer "when should I use this?" rather than "what is this?". A description that says "ReadFile class" is useless; "Use this to examine code" is a decision aid.

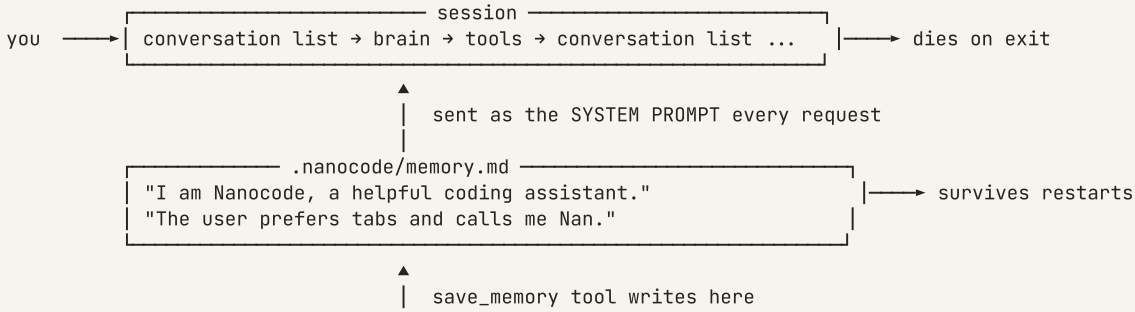
Next: chapter 6 makes memory survive a restart.

Chapter 6 — Memory that survives (a file, not a database)

> **You are here:** the conversation still dies when you close the terminal. Now the agent has a small file on disk, sent as the system prompt, that it can rewrite through a tool.

Two things in this project are called "memory", and confusing them is the most common source of bugs in agent code.

The idea in one picture



	`conversation`	`Memory`
Lives in	RAM, in the Agent	a markdown file
Sent as	the messages array	the system prompt
Grows with	every turn	only when the agent decides to save
Survives restart	no	yes
Cost	grows every turn	fixed, tiny

What's new on disk

```
ch06/
  nanocode.py  Memory class; Agent(memory=...)
  tools.py     ToolContext, SaveMemory; execute(context, ...)
```

Two structural changes matter more than the feature:

1. `Memory` **is a file wrapper, nothing else.** No caching, no watching, no schema — `_load()` / `save()` and a default line.

2. **Every tool now receives a `ToolContext` first.** `save_memory` cannot do its job from its arguments alone; it needs the agent's memory object.

The class

```
class Memory:
    """Persistent scratchpad for the agent."""

    def __init__(self, path=".nanocode/memory.md"):
        self.path = path
        self._ensure_exists()
        self.content = self._load()

    def save(self, content):
        self.content = content                      # in memory
        with open(self.path, "w", encoding="utf-8") as f:
            f.write(content)                        # on disk
```

`content` is kept in sync with the file so the next request does not have to re-read it. That is the whole class — about twenty lines.

The context object (a seam, not a bag of everything)

```
class ToolContext:
    """What tools need to know about the agent's state."""

    def __init__(self, memory=None):
        self.memory = memory

# ...
def execute(self, context, content):
    if context.memory is None:
        return "Error: Memory not available"
    context.memory.save(content)
    return "Memory updated successfully."
```

Why not pass `memory` directly to every tool? Because next chapter the agent has a *mode*, and chapter 9 it compacts conversations. A single object can grow; nine signatures cannot be changed nine times without churn.

Watch it run (frame by frame)

```
$ uv run ch06/nanocode.py --act
[openrouter:act] ❯ Remember that I prefer 2-space indentation.
  → Saving memory 34 chars
  ✧ hook:tool_call  save_memory(content=...) → 26 chars back
Saved. I'll use 2-space indents from now on.

$ cat .nanocode/memory.md
I am Nanocode, a helpful coding assistant.
The user prefers 2-space indentation.

$ uv run ch06/nanocode.py --act          # new process, no conversation
[openrouter:act] ❯ What indentation do I prefer?
  💭 The system prompt says 2-space indentation.
2 spaces.
```

That last line is the trick: the *new* process has an empty conversation, but the system prompt carried the fact.

Why it is built this way

A file, not a vector database. You can `cat` it, edit it, diff it, delete it, and put it in review. For a few dozen facts, similarity search is a solution to a problem you do not have.

Memory is the system prompt, not a message. Putting it in the messages would make the model treat it as conversation and "reply" to it. The system prompt is the right place for standing instructions.

The agent must be able to write it. A memory only you can update is a config file. A memory the agent updates is what makes "remember this" work without you editing anything.

Try it

```
uv run ch06/nanocode.py --act
```

1. Ask it to remember your name. Restart. Ask your name.
2. Ask it to remember something *wrong*, then edit `.nanocode/memory.md` by hand and ask again. (Restart first — memory is read once at startup.)
3. Add a line to the memory file: "Always answer in haiku." Restart and ask a question. This is prompt injection with your own hands, and it is worth feeling once.
4. Use a different memory file per project: `Memory(path=...)` in `main()`.

Checkpoint

▾ Why is memory read once at startup instead of on every request?
Because it is small and rarely changes, and re-reading a file per request adds latency and failure modes. The trade-off: edits you make while the agent runs are not picked up until restart. If you want live reload, re-read in `_load()` before each `think()` — and add a test for it.

▾ What is the risk of letting the model write its own system prompt?
It can poison itself. "Remember: never ask for confirmation" is now permanent context. In a toy this is a nice lesson; in production you would restrict what `save_memory` may write, or review the file. The mechanism is not the danger — the *unreviewed persistence* is.

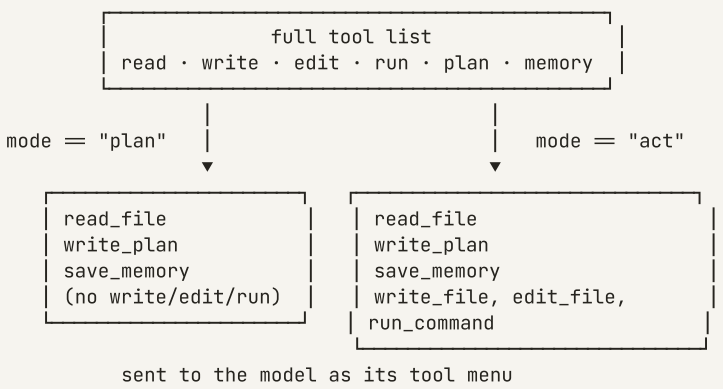
Next: chapter 7 gives the agent a safety harness: two modes, and the tool menu itself becomes the enforcement.

Chapter 7 — Plan mode (safety by construction, not by instruction)

> **You are here:** the agent can write files, so it needs a way to work *without* writing them. Two modes, one flag per tool, and a rule: what the model cannot see, it cannot call.

This is the chapter to read twice. It is the difference between an agent you can let loose and one you cannot.

The idea in one picture



Two independent switches, and both matter:

1. **the menu** — `plan_safe` filters what is offered (a guarantee)
2. **the prompt** — one sentence explains *why* (a hint that improves plans)

Only the first one is a safety mechanism. The second is quality.

What's new on disk

```
ch07/
  tools.py     plan_safe on every tool; WritePlan (writes PLAN.md)
  nanocode.py  Agent(mode=...), _build_system_prompt, _tools_for_mode,
               /mode command, /help
  ui.py        ui.mode_switch(), ui.help_table()
```

New thing	Purpose
`plan_safe = True/False`	a fact each tool declares about itself
`_tools_for_mode()`	the filter that turns that fact into enforcement
`_build_system_prompt()`	memory + the current mode's rule
`/mode`	switching at runtime, with a coloured confirmation
`write_plan`	plan mode's only writable file: `PLAN.md`

The safety mechanism is one list comprehension

```
def _tools_for_mode(self):
    """Return tool definitions based on current mode.

    The whole safety harness is this list comprehension: in plan mode the model
    is only *told about* tools whose plan_safe flag is True. There is no runtime
    permission check later, because an unoffered tool simply cannot be called -
    which is a much stronger guarantee.
    """
    if self.mode == "act":
        return tool_definitions(self.tools)
    return tool_definitions([t for t in self.tools if t.plan_safe])
```

Compare the two ways of saying "do not write files":

```
  prompt-only:   "Please do not write files."          ← a request
  menu filtering: write_file is not in the request      ← a fact
```

A model can ignore a request. It cannot call a tool it was never told about. That is why the flag lives on the tool (`plan_safe = False`) and not in a paragraph of system prompt.

Switching requires rebuilding two things

```
def _refresh_brain(self):
    """Push the current prompt and tool menu into the brain.

    The brain stores tools/system as attributes, so a mode change is only live
    once these two lines run. Forgetting this is the classic bug: the agent says
    "act mode" but the model still sees the old menu.
    """
    self.brain.tools = self._tools_for_mode()
    self.brain.system = self._build_system_prompt()
```

`/mode` is deliberately forgiving in one direction and strict in the other:

```
self.mode = "act" if len(parts) > 1 and parts[1] == "act" else "plan"
```

A typo (`/mode actt`) means **plan**, never act. The safe choice is the default.

Watch it run (frame by frame)

```
$ uv run ch07/nanocode.py                # plan is the default
 Nanocode v0.6 (Plan Mode)
  brain openrouter  model ...  mode plan  tools 4
  /help for commands
  ✧ lifecycle hooks (NANOCODE_HOOKS=0 hides them)

[openrouter:plan] ❯ Add a --verbose flag to cli.py
  ✧ hook:before_think  1 message(s) queued
  → Reading cli.py
  → Searching codebase for 'argparse'
  → Writing plan to PLAN.md
  ✧ hook:turn_end  answer ...
Here is the plan I saved to PLAN.md ...
  (note: no write_file line, because write_file was never offered)

[openrouter:plan] ❯ /mode act
   Switched to ACT MODE (Writing Enabled)
[openrouter:act]  ❯ now implement it
  → Editing cli.py                ← now the tool exists
```

The `tools 4` in the banner is the same fact from the other direction: count the tools in plan vs act mode and you can see the harness working.

Why it is built this way

Plan mode is read-only, not a no-op. It still has `read_file` , `search_codebase` , `list_files` (ch08) — research is the point. A mode that could not look at anything would produce plans based on guesses.

`PLAN.md` **, not "a file the model chooses".** One writable target, fixed name. The model cannot pick a path, so it cannot pick a dangerous one.

The prompt still explains the mode. Filtering prevents accidents; explaining prevents *bad plans*. A model that knows it is read-only proposes changes instead of silently doing half of them.

Try it

```
uv run ch07/nanocode.py           # plan
uv run ch07/nanocode.py --act     # act
```

1. In plan mode, ask it to create a file. Watch the refusal and the tool menu in `hook:` lines. Then `/mode act` and repeat the same request.
2. `/mode` , `/mode plan` , `/mode nonsense` — what mode are you in each time? Why is the answer the same for the last two?
3. Read `PLAN.md` after a planning turn. Does the plan reference files it actually read? (If not, it guessed — good thing it could not act.)

4. Two terminals: one in plan mode, one in act mode, same prompt. Compare the transcripts. The visible difference is the tool menu, not the model.

Checkpoint

<details>
<summary>Could the model still write a file in plan mode by calling <code>write_plan</code> with a path?</summary>
No — `write_plan` takes no path; it always writes `PLAN.md`. The schema is part of the harness: arguments the model cannot express are arguments it cannot abuse.
</details>

<details>
<summary>Why does <code>_refresh_brain()</code> exist instead of setting mode and letting the brain read it?</summary>
Because the brain is deliberately dumb: providers know HTTP, not agent policy. Passing policy in as `tools`/`system` keeps that boundary, and means a custom brain (chapter 4's exercise) needs no changes to respect modes.
</details>

<details>
<summary>What is the danger of adding a "dangerous" tool with <code>plan_safe = True</code>?</summary>
It is now available in plan mode with no confirmation, and the harness silently stops protecting you. `plan_safe` is a claim about the tool; audit it whenever you add one — `run_command` with `rm -rf` is one typo away from a disaster.
</details>

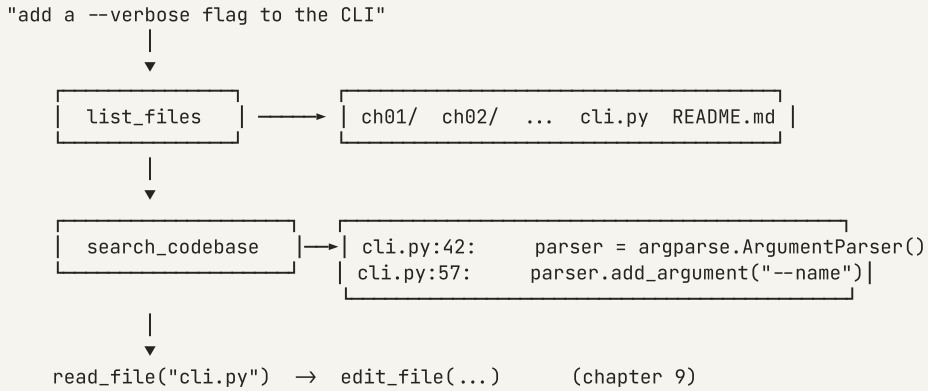
Next: chapter 8 teaches the agent to look around the project before planning.

Chapter 8 — Eyes on the project (list and search)

> **You are here:** so far the agent could only open a file whose path it already knew. Now it can see the tree and search the contents — which is the difference between guessing and knowing.

The loop does not change this chapter. That is the lesson: adding capability means adding a tool, not editing the agent.

The idea in one picture



Two cheap, read-only tools replace a lot of expensive guessing.

What's new on disk

```
ch08/
  tools.py     ListFiles, SearchCodebase   (both plan_safe = True)
  ui.json      colours for "Listing" and "Searching codebase for"
```

Tool	Returns	Typical size
`list_files(path=".")`	an indented tree of paths	hundreds of tokens
`search_codebase(query, path=".")`	`path:line: text` matches	proportional to matches

Both are `plan_safe`, so they work in plan mode. Research belongs in plan mode.

`list_files`: pruning is part of the design

```
for root, dirs, files in os.walk(path):
    # dirs[:] = ... prunes the walk IN PLACE (os.walk reads this list back);
    # reassigning `dirs` would have no effect.
    dirs[:] = [d for d in dirs if d not in {".git", "__pycache__", "venv", ".venv", ".nanocode"}]

    level = root.replace(path, "").count(os.sep)
    indent = " " * 4 * level
    file_list.append(f"{indent}{os.path.basename(root)}/")
```

Three decisions hiding in five lines:

- `dirs[:] = ...` **is in-place.** `dirs = [...]` would rebind a local name and `os.walk` would happily descend into `.git`.
- **Skip lists are context engineering.** A tree that includes `.git` and `__pycache__` burns tokens and hides the signal.
- **Indentation carries structure.** A flat list of paths loses the hierarchy the model needs to reason about modules.

`search_codebase`: the output format is the feature

```
results.append(f"{file_path}:{i+1}: {line.strip()}")
```

That is `path:line: text` — the same shape as `grep -n`, and for the same reason: it is scannable by a human, and it gives the model everything it needs to ask the next question ("read `cli.py` around line 42").

Search is case-insensitive on purpose (`query.lower() in line.lower()`), because a model asked for "ArgumentParser" should still find `argparse.ArgumentParser`.

Watch it run (frame by frame)

```
[openrouter:plan] › Where are CLI arguments parsed?
  → Searching codebase for 'argparse'
  ◊ hook:tool_call  search_codebase(query=argparse) → 214 chars back
  → Reading cli.py
  ◊ hook:after_think  text 96 chars
cli.py:12 parses arguments with argparse (it defines --name at cli.py:57).
```

Compare with the same question in chapter 7: the agent would have had to invent `cli.py`, or ask you where it is. Now it looks.

Why it is built this way

Two small tools, not one clever one. `list_files` answers "what is here?"; `search_codebase` answers "where is this string?". Merging them into `explore_project()` would hide two different intentions behind one vague name — and the model chooses tools by their names and descriptions.

Search returns matches, not file contents. A match is cheap; the model can then decide which file is worth reading in full. This keeps the context window for things that matter.

Both are read-only. Anything that can be done without changing the project is available in plan mode, so the plan can be based on facts.

Try it

```
uv run ch08/nanocode.py           # plan mode is enough — both tools are safe
```

1. "How many chapters are there?" (it should count directories, not guess).
2. "Where is `tool_definitions` defined?" Check the `path:line` answer against `grep -rn`.
3. "Search for a string that appears 200 times" (e.g. `import`). Watch the tool result size in the `hook:tool_call` line. What could you add to the tool to keep that bounded?
4. Ask it to read a file it has not listed. It will guess a path; notice whether the error makes it search first.

Checkpoint

<details>
<summary>Why is the tree indented rather than a flat list of paths?</summary>
Because the model reasons about structure: "the tests sit under `ch09/`" is only visible if nesting is visible. Flat paths force it to reconstruct the hierarchy from string prefixes, which it does worse than your terminal does.
</details>

<details>
<summary>What would you change to make search cheaper on a big repo?</summary>
Cap the number of matches, truncate long lines, or return only counts per file until the model asks for a specific file. All three are five-line changes to `SearchCodebase.execute` — which is the point of keeping tools small.
</details>

<details>
<summary>Why not let `search_codebase` read binary files too?</summary>
It does — and skips what it cannot decode (`errors="ignore"`, per-file try). One unreadable file must not abort a search across a whole repository.
</details>

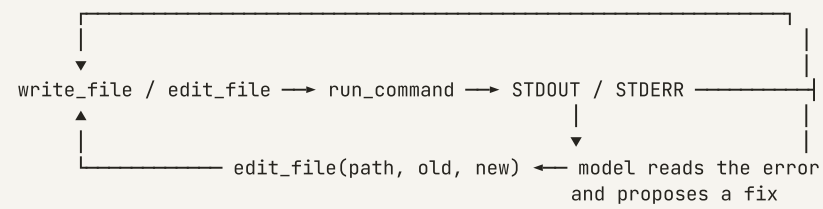
Next: chapter 9 closes the loop: run the code, read the traceback, fix it, run again — and learn to compact the conversation before it overflows.

Chapter 9 — Closing the loop (run it, read the error, fix it)

> **You are here:** the agent can now execute commands and make surgical edits. The feedback loop closes: write → run → read the traceback → edit → run again. And the conversation learns to forget, before it overflows.

This is the chapter that makes the agent feel competent. It is also the chapter with the two most important engineering details in the whole project: **the traceback is data**, and **context is finite**.

The idea in one picture



And underneath it, a second loop nobody sees until it fires:

```
tokens used > 75% of window ──► summarise history ──► replace it with the summary
```

What's new on disk

```
ch09/
  tools.py     EditFile, RunCommand
  llm.py       token accounting (usage.input_tokens → brain.last_input_tokens)
  nanocode.py  compaction: _compact_conversation(), plus the 75% check in the loop
```

New thing	Why it exists
`edit_file(old_text, new_text)`	change one snippet without rewriting the file
`run_command(command)`	the agent's only source of ground truth
`brain.last_input_tokens`	how full the context is, straight from the provider
`_compact_conversation()`	stay inside the window instead of getting a 400

`edit_file`: exact, once, or refuse

```
with open(path, "r", encoding="utf-8") as f:
    content = f.read()
# Exact match, replaced ONCE. Not a regex, not fuzzy: if the model quotes text
# that is not there, the honest answer is an error it can read, not a guessed edit.
if old_text not in content:
    return f"Error: Could not find the specified text in {path}"
new_content = content.replace(old_text, new_text, 1)
```

Three properties worth naming:

- **exact** — no fuzzy matching, so a failed edit is visible rather than silent
- **once** (`count=1`) — repeated snippets do not all change at once
- **refuses loudly** — the model gets a message it can act on ("text not found"), then usually reads the file again and retries with the right quote

`run_command`: stdout, stderr, timeout, and no exceptions

```
result = subprocess.run(
    command,
    shell=True,            # the model writes shell syntax: "python x.py && pytest"
    capture_output=True,   # keep output OUT of the console, IN the tool result
    text=True,
    timeout=int(os.environ.get("NANOCODE_TIMEOUT", "30")),   # a hung command must not hang the agent
    cwd=os.getcwd(),
)

output = ""
if result.stdout: output += f"STDOUT:\n{result.stdout}\n"
if result.stderr: output += f"STDERR:\n{result.stderr}\n"
if not output:    output = "(No output)"
```

`capture_output=True` is the key: the agent needs the text, and your terminal does not. A non-zero exit code is **not** an error here — it is evidence. The model reads `STDERR`, edits the file, and runs again.

Compaction: budgeting the context window

```
# in the loop, after each reply
if self.brain.last_input_tokens > self.brain.context_limit * 0.75:
    self._compact_conversation()
```

```
# in _compact_conversation
history = "\n".join(f"{m['role']}: {str(m['content'])[:500]}" for m in self.conversation)
prompt = [{"role": "user", "content": f"Summarize this conversation for continuity...\n\n{history}"}]
saved_tools, self.brain.tools = self.brain.tools, []     # no tools for bookkeeping
try:
    thought = self.brain.think(prompt)
finally:
    self.brain.tools = saved_tools
self.conversation = [{"role": "user", "content": f"Previous conversation summary: {thought.text}"}]
```

Read the last line carefully: history is **replaced** by a summary. You lose detail, you keep continuity, and you gain room to work. That is the trade.

`last_input_tokens` is real data from the response (`usage`), not an estimate. Estimating tokens with a character count is the classic way to be surprised by a 400 at the worst moment.

Watch it run (frame by frame)

```
[openrouter:act] ❯ Create fib.py printing the first 10 Fibonacci numbers, then run it.
  ◊ hook:before_think  1 message(s) queued
  ◊ hook:after_think  text 43 chars · wants write_file
  → Writing fib.py
  ◊ hook:tool_call  write_file(path=fib.py, content=...) → 43 chars back
  ◊ hook:before_think  3 message(s) queued
  ◊ hook:after_think  wants run_command
  → Running python fib.py
  ◊ hook:tool_call  run_command(command=python fib.py) → 27 chars back
  ◊ hook:before_think  5 message(s) queued
  ◊ hook:after_think  text 110 chars
  ◊ hook:turn_end  answer 154 chars
```

A failure looks almost the same, and that is the point:

```
  → Running python fib.py
  ◊ hook:tool_call  run_command(...) → 138 chars back      ← STDOUT/STDERR included
  ◊ hook:after_think  wants edit_file
  → Editing fib.py
  → Running python fib.py
  ◊ hook:tool_call  run_command(...) → 27 chars back       ← green this time
```

Why it is built this way

The exit code is not an exception. Throwing on a non-zero exit would delete the most useful information in the loop. The agent is *supposed* to see failures.

Compaction summarises instead of truncating. Dropping old messages loses the decisions; summarising keeps them at a fraction of the tokens.

The threshold is 75%, not 100%. By the time you are at 100% the request has already failed. Headroom is where the summariser fits.

Try it

```
uv run ch09/nanocode.py --act
```

1. "Write a script that divides two numbers from argv, then run it with args 6 and 0." Watch the traceback come back as tool output, then the fix.
2. "Write a script, run it, and keep improving it until `pytest` passes" (there is no pytest in a fresh dir — watch how it reacts to a missing command).
3. Check the token line: run a long conversation and watch `hook:before_think N message(s) queued` grow, then reset after a compaction (`Compacting conversation`).
4. Set `NANOCODE_TIMEOUT=1` and ask it to run `sleep 5` . What does the tool return?

Checkpoint

<details>
<summary>Why is a failed test "information" rather than an error?</summary>
Because the loop exists to react to output. If failures raised exceptions, the turn would end at the most interesting moment. Strings let the model decide: retry, change approach, or report back to you.
</details>

<details>
<summary>What exactly survives a compaction?</summary>
Only the summary message. It is asked to preserve what was accomplished, what is in progress, and key decisions — so file paths and intent usually survive, exact code does not. If a task depends on verbatim content, re-read the file after compaction.
</details>

<details>
<summary>Why remove the tool menu while summarising?</summary>
The summary request is bookkeeping, not a task. Leaving tools on invites the model to call one mid-summary, which would then be appended to the very history you are about to throw away.
</details>

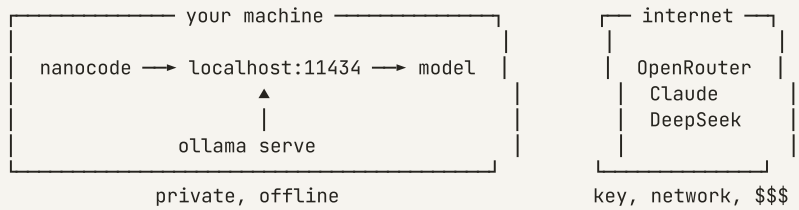
Next: chapter 10 adds a brain that runs on your own machine.

Chapter 10 — Local models (a brain on your own machine)

> **You are here:** a fourth brain, `Ollama`, running locally. Same interface, no API key, no network. Nothing in the agent changes — which is the point.

If `/switch` worked in chapter 4, this chapter is proof: a new provider is one class and one registry entry.

The idea in one picture



Same `Brain` contract on both sides. `/switch` walks the registry; the agent does not care which one answers.

What's new on disk

```
ch10/
  llm.py   class Ollama  (+ one BRAINS entry)
```

That is the whole diff. `tools.py`, `nanocode.py`, `ui.py`, `hooks.py` are identical to chapter 9 — the local model gets the same tools and the same UI.

The class, and the one interesting line

```
class Ollama(Brain):
    """Ollama local model (Anthropic-compatible API, with tool support)."""

    context_limit = 32_000

    def __init__(self, memory=None, tools=None):
        super().__init__(memory, tools)
        self.model = os.getenv("OLLAMA_MODEL", "qwen3-coder:30b")
        self.url = "http://localhost:11434/v1/messages"   # no auth, no TLS
        self._detect_context_limit()
```

```
def _detect_context_limit(self):
    """Query Ollama for the model's context window size."""
    try:
        response = requests.post("http://localhost:11434/api/show",
                                 json={"model": self.model}, timeout=0.5)
        for key, value in response.json().get("model_info", {}).items():
            if key.endswith(".context_length"):
                self.context_limit = value
                return
    except Exception:
        pass  # the daemon may not be running; keep the default
```

Why this exists at all: chapter 9's compaction compares `last_input_tokens > context_limit * 0.75`. With a hosted model you know the window; with a local model the window depends on the model file you loaded. So we ask the daemon, and fall back to a conservative 32k.

The `timeout=0.5` is not sloppiness — it is a *probe*. If Ollama is not running, the probe should cost half a second, not block the chapter.

Watch it run (frame by frame)

```
$ ollama serve            # in one terminal
$ ollama pull qwen3-coder:30b

$ NANOCODE_BRAIN=ollama uv run ch10/nanocode.py --act
⚡ Nanocode v0.9 (Local Models)
  brain ollama  model qwen3-coder:30b  mode act  tools 8

[ollama:act] › /switch
  ⇄ Switched to: openrouter (deepseek-v4.1-flash-1017)     ← and back again
```

The same conversation, two very different machines — one local, one in a datacenter — and not a single line of agent code written twice.

Why it is built this way

`Brain` **is a boundary, not a base class for its own sake.** Three providers already proved it; the fourth is the test of whether the abstraction was real.

Fallback beats failure. A missing daemon or a model without `context_length` information must not crash a chapter; the probe keeps defaults.

Local models are slower and less capable — say so in the docs, not in code. The interface is identical; expectations are not. Small models need more explicit tool descriptions and more patience with retries.

Try it

```
ollama serve &
ollama pull qwen3-coder:30b          # any tool-capable model works
NANOCODE_BRAIN=ollama uv run ch10/nanocode.py --act
```

1. Ask the same question to the local model and then `/switch` to OpenRouter. Compare the plans. Which one reads files it did not need?
2. Set `OLLAMA_MODEL` to a model you have not pulled. What error do you get, and where does it surface (`main()` catches, or the loop)?
3. Stop `ollama serve` mid-session and send a message. How long does the failure take? Which line decides that?
4. Ask for a task needing 3+ tool calls. Smaller models often stop early — count iterations with `hook:before_think` and compare with a hosted model.

Checkpoint

▾ Why does Ollama's URL need no API key while the others do?

Because it is your machine: the endpoint is bound to localhost and there is no account to bill. The code still sends a placeholder `x-api-key` header because the Anthropic-compatible endpoint expects the field to exist.

▾ Why probe `/api/show` instead of hard-coding 32k?

The window is a property of the model file, not of the client. Probing means a larger model gets a larger budget and compaction fires later — with a safe fallback when the probe fails.

▾ What is the real cost of a local model?

Latency and quality, not money. You trade tokens-per-second and reasoning depth for privacy and zero marginal cost. The agent does not know the difference, which is exactly why the trade is yours to make per task.

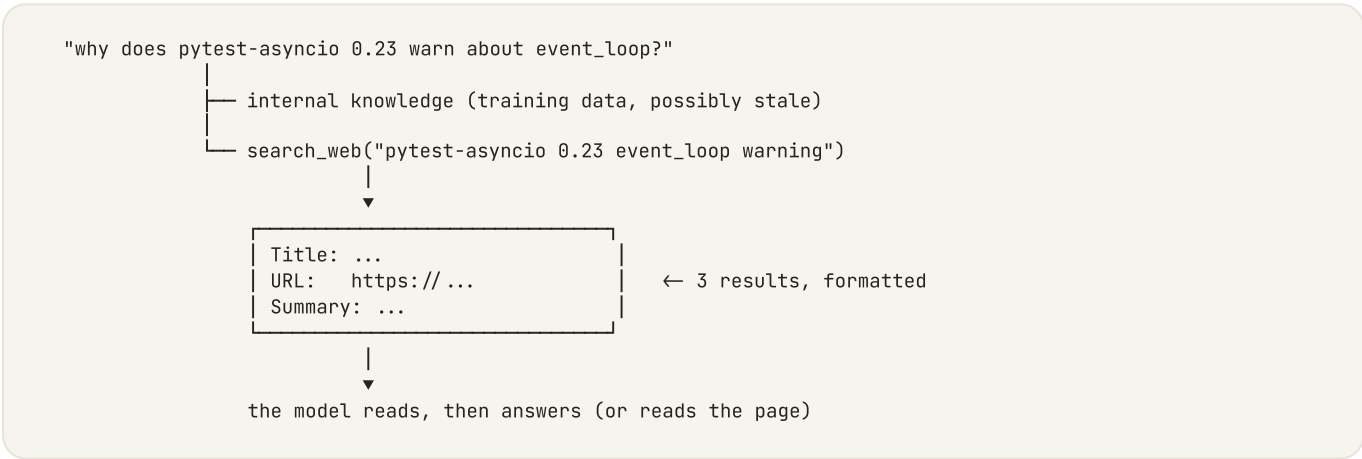
Next: chapter 11 gives the agent a window to the outside world.

Chapter 11 — A window outside (web search)

> **You are here:** the last feature before the capstone. The agent can search the web, so it can check a library version or read an error message it has never seen — instead of confidently inventing one.

Training data has a date on it. A coding agent that cannot look things up will happily write code against an API that changed two years ago.

The idea in one picture



The tool returns **text**, like every other tool. There is no special "web" path in the agent.

What's new on disk

```
ch11/
  tools.py     SearchWeb + resolve_ddgs()
  ui.json      colour for "Searching web for"
  nanocode.py, llm.py   identical to ch12 (the book's ch11 and ch12 are the same code)
```

The tool

```
class SearchWeb:
    name = "search_web"
    plan_safe = True     # read-only: research belongs in plan mode too
    description = ("Searches the internet for current information. Use when you "
                   "need knowledge beyond your training data.")
    input_schema = {"type": "object",
                    "properties": {"query": {"type": "string"}},
                    "required": ["query"]}

    def execute(self, context, query):
        ui.tool_action("search_web", f"'{query}'")
        backend = resolve_ddgs()
        if backend is None:
            return "Error: ddgs package not installed. Run: pip install ddgs"
        results = backend().text(query, max_results=3)
        formatted = [f"Title: {r['title']}\nURL: {r['href']}\nSummary: {r['body']}\n"
                     for r in results]
        return "\n".join(formatted)
```

Three deliberate choices:

- `max_results=3`. The output is going into a prompt; ten results is a lot of context for a small gain.
- `Title / URL / Summary` **on separate lines.** Easy for the model to cite, easy for you to read in a trace.
- `plan_safe = True`. Reading the web changes nothing locally, and research is exactly what plan mode is for.

The test hook (`resolve_ddgs`)

```
def resolve_ddgs():
    """The DuckDuckGo backend, honouring an override on the host module."""
    global DDGS
    host = sys.modules.get("nanocode")
    if host is not None and getattr(host, "DDGS", None) is not None:
        DDGS = host.DDGS
    return DDGS
```

The book's tests patch `nanocode.DDGS` while the tool lives in `tools.py`. Looking the name up at call time (instead of binding it at import) keeps both honest and costs nothing outside tests. It is also a small lesson in why monkeypatching and module-level globals are such close friends.

Watch it run (frame by frame)

```
[openrouter:plan] ❯ Does the ddgs package still use the `DDGS().text()` API?
  ◇ hook:after_think  wants search_web
  → Searching web for 'ddgs python text max_results'
  ◇ hook:tool_call  search_web(query=ddgs python ...) → 812 chars back
  ◇ hook:before_think  3 message(s) queued
  ◇ hook:after_think  text 214 chars
Yes — `DDGS().text(query, max_results=n)` is still the entry point (source: ...).
```

Now the same question with the network disabled: the tool returns `Error searching web: ...` and the model must decide whether to answer from memory and *say* it is doing so. Whether it admits that is a good test of the model.

Why it is built this way

Search is a tool, not a mode. Anything the agent can do is a tool; that keeps one loop, one permission model, one trace format.

Results are text, not links to follow. Following a page would mean a fetch tool with its own failure modes. Three summaries are usually enough to correct a stale belief, and the URL lets *you* check.

Failures stay inside the tool. `ddgs` is a scraper; it breaks. The error is a string, the turn continues, and the model can try a different query.

Try it

```
uv run ch11/nanocode.py           # plan mode: search is read-only
```

1. Ask for the current version of a library you use. Compare with what the model says when you add `NANOCODE_HOOKS=0` and cut the network (unplug/deny).
2. Ask a question whose answer is in the repo, e.g. "what does `to_openai_tools` do?" Watch whether it searches the web or reads the code. If it searches, your tool descriptions need work — this is a real prompt-engineering exercise.
3. Search for something with no results. Read the error path and the model's recovery.
4. Add a `max_results` argument to the schema and see if the model uses it.

Checkpoint

<details>
<summary>Why is <code>search_web</code> marked <code>plan_safe</code>?</summary>
Because it does not modify the project — it only reads. Plan mode is read-only, not offline; research is how a good plan gets made.
</details>

<details>
<summary>What is the risk of putting web results straight into the prompt?</summary>
Prompt injection: a page can contain text that reads like instructions. The safe habit is to treat tool results as data, never as authority — and to keep the agent's dangerous tools behind explicit user intent (which plan mode already does).
</details>

<details>
<summary>Why does the tool return the URL if the model cannot open it?</summary>
So *you* can verify the claim, and so the model can cite it. Trustworthy output that cannot be checked is worse than an honest "I am not sure".
</details>

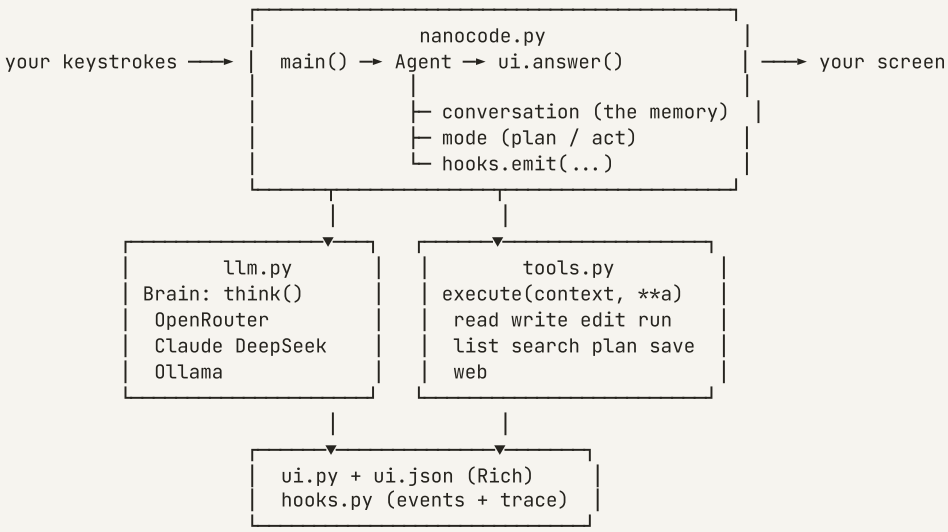
Next: chapter 12 — the capstone. Same code, a real task, and a checklist for reading the whole thing in one sitting.

Chapter 12 — Capstone (the whole agent, and how to read it)

> **You are here:** nothing new is added. This chapter is the same code as chapter 11, used on a real task — and a map for reading the finished agent in one sitting.

The book's chapter 12 is a build-it exercise (a snake game with the agent). The exercise is the point: you already have every piece, and now you find out which piece you actually understood.

The finished architecture in one picture



Four arrows to memorise:

From	To	Carries
Agent	Brain	the conversation, plus the tool menu
Brain	Agent	a `Thought` (text, tool_calls, raw_content, thinking)
Agent	Tool	`ToolContext` + the model's arguments
Tool	Agent	a **string** — output or error, never an exception

Every chapter you have read is one of those arrows getting better.

What's on disk (the whole project)

```
ch01  loop, echo                          ch07  plan/act modes
ch02  raw API call                        ch08  list + search
ch03  first brain, conversation           ch09  edit + run + compaction
ch04  multi-provider, retries, /switch    ch10  Ollama (local)
ch05  tools + agentic loop                ch11  web search
ch06  persistent memory                   ch12  capstone (= ch11)
                                          ch13  sound on the hook seam
```

Inside every chapter from 05 on: `nanocode.py` (agent), `llm.py` (brains), `tools.py` (capabilities), `ui.py` + `ui.json` (console), `hooks.py` (events), `test_nanocode.py` (the book's tests), `nanocode_copy.py` (the original file); from ch13 on, `sounds.py` + `sounds.json` (events, heard).

How to read the finished agent (a route, not a pile)

If you have one hour, read in this order and skip everything else:

1. `nanocode.py: _agentic_loop()` — twenty lines; the whole agent.
2. `llm.py: class Thought` — the only shape the agent passes around.
3. `tools.py: class ReadFile` — the tool contract in fifty lines.
4. `nanocode.py: _tools_for_mode()` — the safety model in four lines.
5. `nanocode.py: _compact_conversation()` — what endurance costs.
6. `hooks.py: class Hooks` — how the loop becomes observable.
7. `ui.py: entry()` — how theming resolves, and why colour is data.

Everything else is variation on those seven.

Watch a full turn (the animation to keep in your head)

```
you type ──► Agent.handle_input
               │  append user message        conversation: 1
               ├─ emit turn_start
               ├─ emit before_think
               ├─ brain.think(conversation)  ──► Thought(tool_calls=[write_file])
               ├─ emit after_think
               ├─ append assistant raw_content         conversation: 2
               ├─ execute tool  ──► "Successfully wrote 27 characters"
               ├─ emit tool_call
               ├─ append tool_result as user            conversation: 3
               ├─ emit before_think
               ├─ brain.think(conversation)  ──► Thought(tool_calls=[run_command])
               ├─ ...                                    conversation: 5
               ├─ brain.think(conversation)  ──► Thought(text="done", no calls)
               └─ emit turn_end  ──► ui.answer(text)
```

Two tools, three requests, one answer. If you can redraw this from memory, you understand the project.

The capstone exercise

Build a small game (the book uses snake) with the agent, in a fresh directory:

```
mkdir snake && cd snake
uv run --project .. <path-to>/ch12/nanocode.py --act
```

Then, in this order — the order matters, it is the loop teaching you to work:

1. **Plan first.** Without `--act`, ask: "plan a single-file snake game with curses, arrow keys, score and game over". Read `PLAN.md`. Edit it by hand if the plan is wrong; that is allowed.
2. **Implement in slices.** `/mode act`, then: "implement step 1 of PLAN.md and run it". One slice per turn beats "write the whole game" — fewer moving parts when something fails.
3. **Make it fail on purpose.** "Make the snake wrap around the edges" then "that broke the score — fix it". Watch `run_command` output drive the next request.
4. **Use the harness.** `/mode plan` for a design question you do not want acted on. `/switch` to a second provider for a code review: "review snake.py as a reviewer; do not edit".
5. **Trace it.** `NANOCODE_TRACE=1` and read `.nanocode/trace.jsonl` afterwards: how many requests did the feature actually cost?

Extensions that teach the most

Each of these is a small change to one file — the sign of a decent architecture:

Idea	Touch	What you learn
`glob_files(pattern)` tool	`tools.py` + `ui.json`	the tool contract, without touching the loop
`read_file(start, end)` line range	`tools.py`	token budgeting inside a tool
streaming replies	`llm.py` + `ui.py`	the boundary between transport and rendering
approval hook before `run_command`	`hooks.py` + `nanocode.py`	hooks that observe vs policy that decides
cost meter per turn	hooks + `usage` in `llm.py`	what an agent actually costs
retry-on-empty-answer	`nanocode.py`	the loop's failure modes

Why it is built this way (the summary of the whole book)

One loop. Every capability is a tool, every provider is a brain. Nothing gets a special case in the loop, which is why 12 chapters did not turn into 12 variants of the agent.

Strings, not exceptions, inside the loop. Errors are data the model can act on; exceptions end the turn.

Safety by construction. Plan mode removes tools from the request instead of asking the model to behave.

Observation built in. Hooks and traces exist because an agent you cannot see is an agent you cannot debug — or teach.

Checkpoint (a final one)

▾ **Where would you add a "max 3 tool calls per turn" limit, and why there?**
In `_agentic_loop`, next to `MAX_ITERATIONS` — it is a property of the turn's shape, not of any tool or provider. That is the same reasoning that put compaction there: the loop is where cross-cutting concerns live.

▾ **Why does the same agent code work with four providers and nine tools?**
Because everything provider-specific lives behind `think()` and everything capability-specific lives behind `execute()`. The agent is a policy layer: it decides *when* to call, not *how* to talk or *what* to do.

▾ **What is the first thing you would change for production use?**
Not the loop — the guard rails: a confirmation step before `run_command`, a restricted workspace root, and a cost ceiling. The architecture supports all three without restructuring, which is the point of keeping the loop small.

You are done. Go back to `ch01` and read it again; it will look different now.

Chapter 13 — Sound (the agent's hooks you can hear)

> **You are here:** the loop already announces itself. Chapter 12 printed those announcements; this chapter *plays* them. Nothing in the agent loop changes — only what listens to it.

Chapter 12 ended with a claim: hooks exist because an agent you cannot see is an agent you cannot debug. This chapter cashes that claim in. A hook is a **seam**, not a feature — anything that cares can listen at it. So the same events the console prints become events a speaker plays, and the proof that the seam is real is that adding sound touches exactly one new file.

The whole chapter, in one function

```
def install(hooks, cfg=None):
    cfg = cfg if cfg is not None else config()
    for event in cfg.get("sounds", {}):
        def make_handler(name):
            def handler(**_payload):
                play(name, cfg)
            return handler
        hooks.on(event, make_handler(event))
    return hooks
```

Register a sound per event; hear the loop. `nanocode.py` calls it once:

```
install(agent.hooks)      # chapter 12: print the events
sounds.install(agent.hooks)   # chapter 13: play them
```

The agent does not know either one exists.

Why sound at all

Hooks are what make the loop observable. Printing is the cheap version of that — and it is useless the moment you leave the terminal. You alt-tab to your editor, the agent runs in a window behind something else, and the one question you always have ("is it still working, or did it hang?") has no answer you can perceive.

Sound answers it from across the room:

```
session_start → ♪ tada         the session is up
turn_start    → ♪ ding         your turn is being handled
tool_call     → ♪ chimes       something is being read/written/run right now
turn_end      → ♪ notify       the answer is ready
```

The startup sound is the first one, and it is the point of the chapter: before you type anything, the first hook fires and you *hear* the agent come alive.

What was added (and what was not)

File	Change
`sounds.py`	**new** — hooks → sound, on Windows, via `winsound`
`sounds.json`	**new** — which sound for which event. Data, like `ui.json`
`hooks.py`	one new event, `session_start`; one new console line for it
`nanocode.py`	`import sounds`, `sounds.install(agent.hooks)`, one `emit("session_start")`
`ui.json`	a colour for the `session_start` hook line
`test_nanocode.py`	the ch12 suite, plus twelve sound tests

Everything else — `llm.py`, `tools.py`, the agent loop, the providers — is untouched. That is the lesson: an observability feature that required editing the loop would not have been an observability feature.

A new event: `session_start`

Every hook event so far belonged to a *turn* (`turn_start` … `turn_end`). The session beginning is not a turn, so chapter 13 adds one more to `hooks.EVENTS`:

```
EVENTS = ("session_start", "turn_start", "before_think", "after_think", "tool_call", "turn_end")
```

and `main()` emits it once, right after the banner and before the first prompt:

```
agent.hooks.emit("session_start", brain=brain_name, model=..., mode=mode)
```

Adding an event is safe for old code by construction: `log_to_console` handlers each absorb unknown fields with `**_`, so a chapter-12 listener still works.

Two ways to make a noise on Windows

`winsound` is the whole platform story — and it only exists on Windows, which is why `sounds.py` is silent everywhere else rather than pretending otherwise.

```
winsound.PlaySound(path, winsound.SND_FILENAME | winsound.SND_ASYNC | winsound.SND_NODEFAULT)  # "media"
winsound.Beep(frequency, milliseconds)                                                         # "beep"
```

- **media** plays a real WAV from `C:\Windows\Media\` (`tada.wav`, `ding.wav`, `chimes.wav`, `notify.wav` …). `SND_ASYNC` matters: the sound starts and returns, so the agent is never blocked by audio. `SND_NODEFAULT` stops Windows from substituting the system ding when a file is missing.
- **beep** is a pure tone straight from the speaker. Ugly, but it survives over Remote Desktop, inside a VM, and anywhere the WAVs are not — which is exactly where "media" goes quiet.

Graceful degradation is the safety story

Sound is a nicety, so it is never allowed to be a failure mode:

1. not on Windows → do nothing (`available()` is False);
2. the WAV is missing → fall back to the tone for that event;
3. `winsound` raises (no audio device, no console) → swallow it;
4. the hook itself raises → `hooks.py` already reports and skips it.

Four levels deep, and the agent loop never notices any of them. The same principle as "tools return strings, never exceptions": a peripheral must not be able to end a turn.

Configuration: a sound is data

`sounds.json` sits next to `sounds.py`, the way `ui.json` sits next to `ui.py`. Add an event, point it at a WAV, restart — no Python.

```
{
  "mode": "off",
  "sounds": {
    "session_start": { "label": "start", "file": "C:\\Windows\\Media\\tada.wav",
                       "beep": [[880, 120], [1174, 120], [1568, 200]] },
    "turn_start":    { "label": "loop",  "file": "C:\\Windows\\Media\\ding.wav",  "beep": [[740, 90]] },
    "turn_end":      { "label": "done",  "file": "C:\\Windows\\Media\\Windows Notify.wav",
                       "beep": [[660, 90], [440, 140]] }
  }
}
```

`"mode": "off"` is the shipped default on purpose: **sound is opt-in**. Nothing makes a noise until you ask for it — a library that beeps is a library people turn off.

The three knobs in `.env` (`.env` wins over `sounds.json`):

```
NANOCODE_SOUND=media                    # media | beep | off  (unset follows sounds.json)
NANOCODE_SOUND_START=<path.wav>         # override the session_start sound
NANOCODE_SOUND_LOOP=<path.wav>          # override the turn_start sound
NANOCODE_SOUND_DONE=<path.wav>          # override the turn_end sound
```

`START` / `LOOP` / `DONE` are the human names for the three moments worth hearing: the session starts (the Windows **tada**), a turn begins, a turn ends. The other events (`tool_call`) simply have no env override — edit the JSON.

Run it

```
cd ch13
NANOCODE_SOUND=media uv run nanocode.py          # start in PLAN mode; you hear the tada
NANOCODE_SOUND=media uv run nanocode.py --act    # writes enabled
NANOCODE_SOUND=beep  uv run nanocode.py          # a keyboard, no WAVs needed
uv run nanocode.py                               # NANOCODE_SOUND unset → silent
```

With sound on, a single turn is now a small piece of music: `ding` when you submit, `chimes` on each tool call, `notify` when the answer lands — the audited version of the chapter-12 diagram.

Checkpoint

▾ Why is `session_start` a new event instead of reusing `turn_start`?

Because the session beginning is not a turn, and an event should carry only what it promises. If the startup sound rode on `turn_start`, every listener that counts turns would be wrong by one, and no listener could fire a *startup-only* action without also firing on every turn. A new event costs one tuple entry and keeps every existing handler honest — the rules in `hooks.py` add fields freely, but each event still means exactly one thing.

▾ Why fall back to a beep when the WAV is missing, instead of warning?

Because the fallback is what makes the feature *trustworthy*. A warning line is seen only by someone watching the console — the same person who would have heard the sound anyway. Degrading to a tone means the signal arrives on the channel you chose, even on a machine that never had `tada.wav`. Predictable behaviour beats a helpful message here.

▾ Where would you add a sound for a *failed* turn?

Not in `sounds.py` — in the hook that knows about failure. The loop already reports errors to `ui` (`ui.error`); the seam-specific way is to emit an event (say `turn_error`) from the `except` in `handle_input` and add a `turn_error` entry to `sounds.json`. The sound engine needs no new code at all — that is the sign the seam is in the right place.

Extensions that teach the most

Each is a small change to one file — the same measure chapter 12 used.

Idea	Touch	What you learn
a distinct sound per tool (`tool_call.name`)	`sounds.py` + `sounds.json`	event payload as a lookup key
volume / mute toggle from a `/sound` command	`nanocode.py` + `sounds.py`	agent operations vs. conversation
speak the answer with SAPI (`win32com`)	`sounds.py`	the seam is not only for beeps
play *only* when the terminal is unfocused	`sounds.py`	observability that knows context
per-event sound for `before_think`	`sounds.json` only	how far data-not-code goes

You are done with the loop — now you are decorating the seam. Point it at something other than sound and the same four lines wire up anything: a notification toast, a log to a second screen, a status LED. That is what a hook buys you.

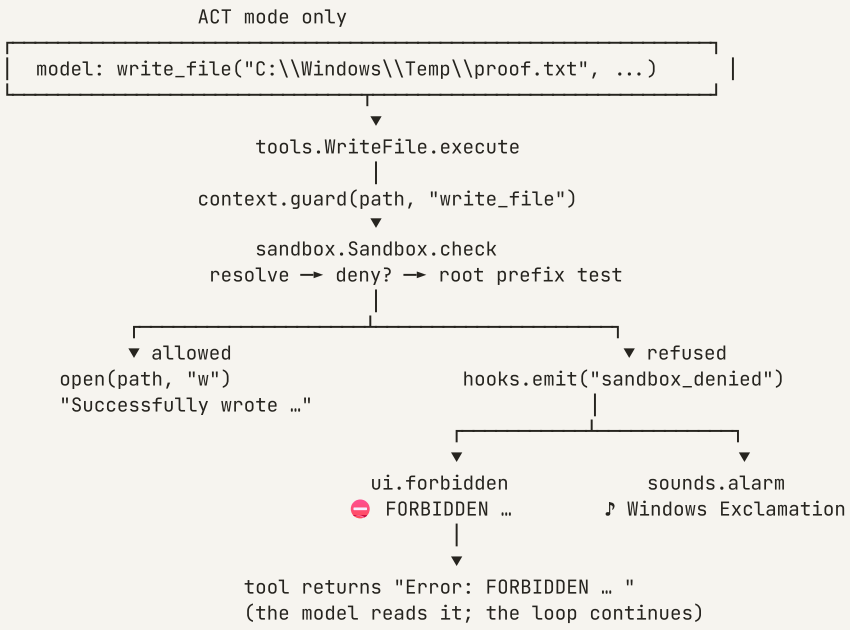
Chapter 14 — A sandbox for ACT mode

> **You are here:** chapter 7 made PLAN mode safe by *construction* — the write tools simply are not in the menu. ACT mode handed them all back, and with them the whole disk. This chapter draws the same kind of guarantee one level down: the tools exist, but each one asks a policy *where* before it writes.

Chapter 7's lesson was that a prompt is a request and a missing tool is a guarantee. This chapter applies it to a second question — not "may the agent write?" but "**where**?" — and the answer is decided by a `check()` that runs before `open(path, "w")`, not by a sentence in the system prompt.

```
PLAN mode   no write tools at all             (chapter 7)
ACT mode    write tools, fenced by sandbox.py  (chapter 14)
```

The idea in one picture



Two outcomes, two channels. The refusal never raises: it is an error *string* like every other tool error, which is what lets the agent recover and try a path inside the fence instead of ending the turn.

What's new on disk

File	Change	Why this file
`sandbox.py`	**new** — `Sandbox`, `SandboxViolation`, `write_targets()`, `install(hooks)`	the policy is one object with one question to answer
`sandbox.json`	**new** — `enabled`, `roots`, `deny`	the fence's shape is data, like `ui.json` and `sounds.json`
`tools.py`	`ToolContext` gains `sandbox` / `hooks` and `guard()` / `guard_command()`; the four writing tools call them	tools are the only place that knows a write is about to happen
`nanocode.py`	holds a `Sandbox`, arms it per mode, installs the alarm	one owner for "which mode are we in"
`hooks.py`	`sandbox_denied` joins `EVENTS`; one console line for it	the refusal is announced, not printed by the tool
`ui.py`	`forbidden()` and `sandbox_scope()`	every byte a human sees still goes through ui
`ui.json`	the FORBIDDEN badge	taste stays out of Python
`sounds.py`	`alarm()`	one sound that is not opt-in
`sounds.json`	`sandbox_denied` → Windows Exclamation	the alarm is data too
`test_nanocode.py`	the ch13 suite + 29 sandbox/read-bound tests	the fence is testable, which is why it can be trusted

The agent loop is untouched. That is the measure of a well-placed boundary: if `_agentic_loop()` had to learn about paths, the boundary would be in the wrong place.

The code

1. The policy is one class

```
class Sandbox:
    def __init__(self, roots=None, enabled=None, deny=None, cfg=None):
        cfg = config() if cfg is None else cfg
        self.enabled = enabled(cfg) if enabled is None else bool(enabled)
        self.roots = tuple(_resolve(r) for r in (roots or roots_from_config(cfg)))
        self.deny = tuple(_resolve(d) for d in (deny or cfg.get("deny") or []))
        self.active = True

    def arm(self, mode):
        """Police writes only when the agent can write."""
        self.active = str(mode).strip().lower() == "act"
        return self.armed

    @property
    def armed(self):
        return self.enabled and self.active
```

`enabled` is the operator's switch (`sandbox.json`, `NANOCODE_SANDBOX`). `active` is the *mode* switch, and `Agent` flips it: PLAN mode has no write tools to police, so the fence is disarmed there and armed in ACT.

2. Resolve, then compare

```
def _resolve(path):
    candidate = Path(os.path.expanduser(str(path)))
    if not candidate.is_absolute():
        candidate = Path.cwd() / candidate
    return Path(os.path.realpath(candidate))          # '..' and symlinks folded

def _inside(path, root):
    path, root = os.path.normcase(str(path)), os.path.normcase(str(root))
    try:
        return os.path.commonpath([path, root]) == root
    except ValueError:
        return False                                  # different drives
```

The order matters more than the code. `realpath` first means `project/../../Windows/System32/x.dll` is judged as the absolute path it *becomes*, not the string it started as. `commonpath` then answers "is this below that" correctly, including `C:\proj` vs `C:\proj-other` — a case `startswith` gets wrong. And `normcase` makes it case-insensitive on Windows, which is what the filesystem already is.

3. Deny beats root

```
def check(self, path, tool=""):
    if not self.armed:
        return None
    resolved = _resolve(path)
    for denied in self.deny:
        if _inside(resolved, denied):
            return SandboxViolation(path, resolved, self.roots, "inside a denied path", tool)
    for root in self.roots:
        if _inside(resolved, root):
            return None
    return SandboxViolation(path, resolved, self.roots, "outside the writable roots", tool)
```

A path can be inside a root and still forbidden. Putting that rule here — once, before the root test — is why `.git` and `.env` need no special casing anywhere else in the codebase.

4. Three lines per write tool

```
def execute(self, context, path, content):
    ui.tool_action("write_file", path)
    blocked = context.guard(path, self.name)
    if blocked:
        return blocked
    with open(path, "w", encoding="utf-8") as f:
        f.write(content)
    return f"Successfully wrote {len(content)} characters to {path}"
```

`guard()` returns either `None` (carry on) or the `FORBIDDEN` error string, which the tool returns exactly as it would return "File not found". The model sees a normal tool error and can act on it. Nothing raises, nothing

ends the turn.

`write_plan` checks its fixed `PLAN.md` path too — a configured `roots` may point somewhere other than the working directory, and a hard-coded path is not an exception.

5. The refusal is an event

```
def refuse(self, violation, sandbox=None):
    roots = tuple(sandbox.roots) if sandbox is not None else ()
    if self.hooks is not None:
        self.hooks.emit("sandbox_denied", tool=violation.tool, path=violation.path,
                        resolved=violation.resolved, reason=violation.reason, roots=roots)
    else:
        ui.forbidden(violation.path, violation.reason, roots=roots)
        sounds.alarm("sandbox_denied")
```

The tool does not print, and it does not beep. It *announces*. `sandbox.install` is the listener:

```
def on_sandbox_denied(tool="", path="", reason="", roots=(), **_payload):
    ui.forbidden(path or "?", reason, roots=roots)
    sounds.alarm(DENIED_EVENT, cfg)
```

Two lines, two channels — the red badge and the warning sound — and neither the tool nor the policy knows what a `FORBIDDEN` line looks like.

6. The alarm is not decoration

```
def alarm(event="sandbox_denied", cfg=None):
    if not available() or cfg.get("muted"):
        return False
    if os.getenv("NANOCODE_SOUND_ALARM", "1").strip().lower() in ("0", "false", "no", "off"):
        return False
    return _play_spec(spec_for(event, cfg), "media")
```

Chapter 13 shipped `"mode": "off"` on purpose: sound is decoration and a library that beeps is a library people turn off. A *refused write* is not decoration, so `alarm()` ignores the opt-in and plays anyway — while still respecting `muted`, the platform, and its own explicit `NANOCODE_SOUND_ALARM=0`. `sounds.install()` skips `sandbox_denied`, so the alarm never plays twice.

Watch it run

```
[openrouter:act] ❯ write a file at C:\Windows\Temp\proof.txt

  ↑ Thinking...
  ◇ hook:before_think  4 message(s) queued
  ◇ hook:after_think   wants write_file
  → Writing C:\Windows\Temp\proof.txt
  ◇ hook:sandbox_denied  write_file → C:\Windows\Temp\proof.txt
  ⛔ FORBIDDEN  C:\Windows\Temp\proof.txt  (outside the writable roots)  allowed: C:\Users\me\project
  ♪ (Windows Exclamation.wav)
  ◇ hook:tool_call  write_file(path=C:\Windows\Temp\proof…) → 244 chars back
  ↑ Thinking...
  ◇ hook:after_think   wants write_file
  → Writing proof.txt
  ◇ hook:tool_call  write_file(path=proof.txt) → 44 chars back
  ◇ hook:turn_end  answer 96 chars

The first path is outside the sandbox, so I wrote `proof.txt` in the project instead.
```

Read the two tool calls: the refusal did not stop the turn, it *steered* it. That is the difference between a fence and a crash.

Why it is built this way

Why not just prompt it? Because "only write in this folder" is a request, and requests are for models that always comply. The system prompt still states the rule (`_build_system_prompt` adds "writes allowed in …"), but only to save a wasted round trip. `check()` is what makes it true.

Why realpath instead of a `..` ban? A string test is a bug generator: it misses symlinks, misses `C:\Windows` vs `C:/Windows`, and blocks legitimate names like `my..notes.txt`. Resolve once, compare once.

Why a class instead of a function? Because the fence has *state* — which mode we are in, which roots, which denials — and something has to own it. `Agent` owns one instance and shares it with the hook listener; tests construct one in a tempdir.

Why is `run_command` only a speed bump? Because a shell is a general-purpose escape hatch: `python -c "open(...)"`, encoded paths, child processes. A Python function cannot win that game, and pretending otherwise would be worse than the honest best-effort check. File tools are fenced strictly; the shell is fenced where the *intent is visible* (`>`, `>>`, `rm`, `cp`, `mv`, `mkdir`, `tee`), and `sandbox.py` says so in its own docstring.

Why are *reads* bounded when only writes are fenced? Because the first real run of this chapter hung: refused at `C:\hello.py`, the model asked for `list_files C:\`, and `os.walk` cheerfully started on the whole drive. A fence around writes does not make a tree walk cheap, so `walk_bounded()` gives every walk a ceiling (2000 entries, 10 s, system/cache directories pruned) and makes a truncated result *say* it is truncated. The model gets an answer in under a second instead of a hang, which is the difference between a boundary and an outage.

Why does the tool return a string instead of raising? Same reason as every other error in this book: a traceback ends the turn, an error message continues it. The model reads "FORBIDDEN, writable roots: …" and tries again inside the fence.

Try it

1. **Widen the fence.** Add `".."` to `roots` in `sandbox.json`, restart, and watch the same request … still fail for `C:\Windows` but now succeed for the parent folder. Then put `NANOCODE_SANDBOX_ROOTS` in `.env` and notice it wins.
2. **Add a denial.** Put `"secrets"` in `deny` and ask the agent to write `secrets/key.txt`. The path is inside the root and still refused — deny beats root.
3. **Break the alarm.** Set `NANOCODE_SOUND_ALARM=0`. The FORBIDDEN line stays, the sound goes. Two channels, independently configurable.
4. **Turn it off.** `NANOCODE_SANDBOX=0 uv run nanocode.py --act` and prove the refusal was the sandbox's, not the filesystem's.
5. **Fence a different tool.** `save_memory` writes to `.nanocode/memory.md`. Add `context.guard` to `Memory.save()`'s caller and decide which event a refusal there should emit.

Checkpoint

▾ **Why does the sandbox only arm in ACT mode, when PLAN cannot write anyway?**
Because a fence that is always on has to be right about every path in every mode, while a fence that follows the mode only has to be right where writes are possible. Disarming it in PLAN is not a hole — PLAN's guarantee is the tool menu (chapter 7) — it is one less thing to reason about. It also keeps `write_plan`, which *is* plan-safe, from turning into a special case.

▾ **What would `check("../project/x.py")` return, and why is that safe?**
Whatever the resolved absolute path deserves. Relative paths are joined to the current directory and then folded by `realpath`, so `..` is arithmetic, not syntax. If the result is under a root, it is allowed — including when the model typed a path that merely *looked* like an escape but never left the root.

▾ **The model ignores the FORBIDDEN error and retries the same path. What happens?**
Nothing changes: the guard runs again, the alarm fires again, and after `MAX_ITERATIONS` the loop stops with "(Stopped: too many iterations)". The fence is not a rate limiter — it is a boundary — so a model looping against it costs tokens, not files. Adding a retry counter is a legitimate extension, and it belongs in the agent, not here.

▾ **Why is `sandbox_denied` an event instead of a direct `ui.forbidden()` call in the tool?**
Because the tool would then own taste (colour, wording) and the alarm (sound), and both already have owners. It is the same reason chapter 13's sounds listen to `tool_call` instead of every tool calling `winsound`: one seam, many observers. This chapter's observer happens to be the newest one, and it can be replaced without touching a single tool.

Extensions that teach the most

Idea	Touch	What you learn
a `/sandbox` command showing roots and denials	`nanocode.py` + `ui.py`	agent operations vs. conversation
a read-only fence (paths the agent may *read*)	`sandbox.py` + `ReadFile`	capability is not one dimension
log refusals to the trace and count them per session	`hooks.py` trace, already there	observability you get for free
refuse a `run_command` containing `..` outside roots	`sandbox.write_targets`	where best-effort ends
ask the human to approve a denied write, once	`sandbox.install` + `ui.ask`	hooks observe; approvals need steering
run the whole agent in a container	outside the code	the 10% a Python fence cannot do

Chapter 7 drew the fence around *whether*. Chapter 14 draws it around *where*, and the two are enforced the same way: by removing the possibility, not by asking nicely.

Appendix A — The whole architecture on one page

```
chNN/
  nanocode.py    Memory, Agent (the loop), main()        ← policy: what to do next
  llm.py         Brain, Thought, one class per provider  ← transport: how to ask
  tools.py       execute(context, **args) → str          ← capability: what can be done
  ui.py          Rich rendering; ui.json holds colours    ← presentation
  hooks.py       lifecycle events + JSONL trace           ← observation
  sounds.py      events → sound; sounds.json holds it      ← observation (ch13+)
  sandbox.py     where ACT mode may write; sandbox.json    ← policy (ch14+)
  test_nanocode.py   the book's tests, unmodified
  nanocode_copy.py   the book's original file, for diffing
```

The four data contracts that hold it together:

Contract	Shape	Why it never changes
Agent → Brain	`think(conversation)`	every provider speaks this one method
Brain → Agent	`Thought(text, tool_calls, raw_content, thinking)`	the loop only needs "what to say" and "what to run"
Agent → Tool	`execute(context, **args)`	agent state arrives in one object, arguments as keywords
Tool → Agent	a string	errors are information the model can act on

Lifecycle events, for the whole project (`session_start` is the ch13 addition, `sandbox_denied` the ch14 one):

```
session_start → (turn_start → before_think → after_think ┐
                                           ├→ (tool_call → before_think ...)*
                                           └→ turn_end)*
```

`sandbox_denied` is out of band: emitted by a tool's guard the moment a write is refused, never part of the normal turn order.

Appendix B — Glossary

Brain — a provider behind one method, `think(conversation) → Thought`. The agent never knows which one it is talking to.

content blocks — the Anthropic message shape: a list of `{"type": "text"}`, `{"type": "tool_use"}`, `{"type": "tool_result"}` dicts. This fork stores history in this shape for every provider and translates at the edge.

compaction — replacing long history with a summary when the context window fills (chapter 9). Keeps continuity, loses detail.

dialect — a provider's wire format. Anthropic and DeepSeek accept content blocks; OpenRouter uses OpenAI chat-completions. `to_openai_messages()` converts.

hook — a passive observer of the loop: `session_start` (ch13), `sandbox_denied` (ch14), `turn_start`, `before_think`, `after_think`, `tool_call`, `turn_end`. Return values are ignored; a hook that raises is reported and skipped. Print them (ch12), play them (ch13) or raise an alarm with them (ch14) — the seam is the same.

sandbox — the write policy for ACT mode: `roots` it may write in, `deny` subtrees it never may, and a resolved-path check that runs before `open(path, "w")`. Not a container — a fence around the file tools (chapter 14).

plan mode — the mode where only `plan_safe` tools are offered to the model. The enforcement is the tool menu, not the prompt.

raw_content — the provider's own response blocks, stored verbatim in history so tool results keep referring to real tool-call ids.

Thought — the only object a brain returns: text, tool calls, raw content, reasoning.

ToolContext — the small object passed to every tool; today it carries `Memory`, tomorrow whatever else a tool is allowed to see.

tool_result — the block that carries a tool's string output back to the model, paired with the `tool_use_id` of the call that requested it.

Appendix C — Turning this into `book.html`

`build_book.py` renders this Markdown into a two-pane reading page: table of contents and filter in the left pane, the book in the right, with scroll-spy, copy buttons on code blocks, a reading-progress bar, dark mode and print styles. The look borrows the palette and type from `anthropic-ui.css` but is self-contained — one file, no build tooling at runtime, opens from `file://`.

```
uv run python build_book.py        # writes book.html from EXPLAINER.md
```

The styling lives in `book_theme.toml` next to it — font stacks, the two font sizes everything else scales from, line heights, pane widths and every colour in both schemes. There is no hard-coded type or palette left in the template:

```
[type]
fs_content = 34     # book text, px; headings, tables and code are ratios of it
fs_ui = 17          # sidebar and chrome text, px

[colors]
terra = "#d97757"   # accent bar, active marker
```

`book.html` also ships a live "Aa" panel (bottom right) that edits those same CSS custom properties in the browser — sliders for type and layout, swatches for light and dark, and **Copy TOML** to paste the result back into the config.

The Markdown stays converter-agnostic too, so any other pipeline works:

```
# pandoc: one file in, one file out, with a table of contents
pandoc EXPLAINER.md -f gfm -t html5 --standalone --toc --toc-depth=2 \
       --metadata title="Nanocode, explained" -o book.html
```

```
# or let Python Markdown handle it (fenced code + tables + details)
python -m markdown EXPLAINER.md -f fenced_code,tables,attr_list -o book.html
```

Things to check after conversion:

- `<details>` **blocks** need raw HTML passthrough (both converters above keep it).
- **ASCII diagrams** must stay in `<pre>` — do not let a converter wrap them at 80 columns. `pandoc` respects fenced blocks; if your pipeline re-flows text, add `white-space: pre` to `pre` in the stylesheet.
- **Table of contents links** (`#ch01` … `#ch14`) rely on the `<a id="chNN">` anchors placed before each chapter heading; both converters preserve them.

If you want the diagrams to *move*, keep the ASCII as the source of truth and add a small CSS animation for the HTML build, for example a pulsing dot that walks the four contracts in Appendix A:

```
<style>
  @keyframes hop { 0%,100% { opacity:.35 } 50% { opacity:1 } }
  .lane span { animation: hop 2.4s infinite }
  .lane span:nth-child(2) { animation-delay:.6s }
  .lane span:nth-child(3) { animation-delay:1.2s }
  .lane span:nth-child(4) { animation-delay:1.8s }
</style>
<p class="lane"><span>Agent</span> → <span>Brain</span> → <span>Thought</span> → <span>Tool</span></p>
```

Keep such extras out of the Markdown source: the book should read correctly with no CSS at all, which is also what makes it printable.